RT-Thread PWM Developer Guide

ID: RK-KF-YF-C32

Release Version: V1.1.0

Release Date: 2025-07-30

Security Level: ☐Top-Secret ☐Secret ☐Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Pulse Width Modulation (PWM) functionality is very common in embedded systems. It is a very effective technology that uses the digital output of microprocessors to control analog circuits and is widely used in many fields from measurement, communication to power control and conversion. This document mainly introduces the basic characteristics and usage methods of PWM on the Rockchip platform.

Product Version

Chipset	RT-Thread Version
RK2108	lts-v3.1.x
RK3308	lts-v3.1.x/lts-v4.1.x
RK1808	lts-v3.1.x
RV1109/RV1126	lts-v3.1.x
RK356X	lts-v3.1.x
RK3588	lts-v4.1.x
RV1103/RV1106	lts-v3.1.x
RK3562	lts-v4.1.x
RK3576	lts-v4.1.x
RK2118	lts-v4.1.x
RK3506	lts-v4.1.x
RV1103B	lts-v3.1.x
RK1820	lts-v5.1.x
RK2116	lts-v4.1.x

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Ding Lingxun	2024.11.22	Initial release
V1.1.0	Ding Lingxun	2025.07.30	Added RK1820/RK2116 related instructions and optimized "Function Support" section HTML table

Contents

RT-Thread PWM Developer Guide

1. Feature Support
2. Driver Directory
3. Configuration Guide
4. Software Application
   - 4.1 Framework
   - 4.2 Continuous Output Mode
   - 4.3 Oneshot
   - 4.4 Capture
   - 4.5 Global Control
   - 4.6 Output Offset
   - 4.7 Counter
   - 4.8 Frequency Meter
   - 4.9 IR Input

1. Feature Support

SOC	PWM Version	PWM Features											
		Channel Number	Continuous	Oneshot	Capture	Global Control	Output Offset	Counter	Frequency Meter	IR Output	IR Input	Wave Generator	Biphasic Counter
RK2108	v2	4	Y	Y	Y	N	N	N	N	N	Y	N	N
RK3308	v2	4	Y	Y	Y	N	N	N	N	N	Y	N	N
RK3288	v2	4	Y	Y	Y	N	N	N	N	N	Y	N	N
RK322X/ RK312XH	v2	4	Y	Y	Y	N	N	N	N	N	Y	N	N
RK356X	v2	4	Y	Y	Y	N	N	N	N	N	Y	N	N
RK3588	v2	4	Y	Y	Y	N	N	N	N	N	Y	N	N
RV1103/ RV1106	v3	4	Y	Y	Y	N	Y	Y	N	Y	Y	N	N
RK3562	v3	4	Y	Y	Y	N	Y	Y	N	Y	Y	N	N
RK3576	v4(PWM0)	2	Y	Y	Y	Y	Y	N	N	N	Y	N	N
	v4(PWM1)	6	Y	Y	Y	Y	Y	Y	Y	N	N	Y	Y
	v4(PWM2)	8	Y	Y	Y	Y	Y	N	N	Y	Y	N	Y
RK2118	v4(PWM0)	4	Y	Y	Y	Y	Y	N	N	N	Y	Y	N
	v4(PWM1)	4	Y	Y	Y	Y	Y	Y	Y	Y	N	N	N
RV1103B	v4(PWM0)	4	Y	Y	Y	Y	Y	Y	Y	N	Y	N	N
	v4(PWM1)	4	Y	Y	Y	Y	Y	N	N	N	N	N	N
	v4(PWM2)	4	Y	Y	Y	Y	Y	N	N	N	N	N	N
RK3506	v4(PWM0)	4	Y	Y	Y	Y	Y	N	N	N	Y	Y	N
	v4(PWM1)	8	Y	Y	Y	Y	Y	Y	Y	Y	N	N	Y
RK1820	v4(PWM0)	8	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	N
RK2116	v4(PWM0)	4	Y	Y	Y	Y	Y	N	N	N	Y	Y (channel1~3)	N
	v4(PWM1)	4	Y	Y	Y	Y	Y	Y	Y	N	N	N	N

- PWM v2
  - Each controller has 4 channels;
  - Continuous/oneshot/capture: Any channel of each controller can support;
  - IR input: Only channel 3 of each controller can support.
- PWM v3:
  - Counter/output offset: Any channel of each controller can support;
  - Other functionalities are the same as PWM v2.
- PWM v4:
  - Different controllers may have different numbers of channels, with a maximum of 8 channels supported by a single controller;
  - Continuous/oneshot/capture/global control/output offset: Any channel of each controller can support;
  - Counter/frequency meter/IR output/IR input: Some controllers can support, and only one channel of a single controller can enable the above functions at the same time;
  - Wave generator: Some controllers can support, and multiple channels of a single controller can enable this function at the same time;

- Biphasic counter: Some controllers can support, and multiple channels of a single controller can support this function.

2. Driver Directory

RT-Thread PWM Device Framework:

```
components/drivers/include/drivers/rt_drv_pwm.h
components/drivers/misc/rt_drv_pwm.c
```

PWM Device Driver for Rockchip Platform:

```
bsp/rockchip/common/drivers/drv_pwm.c
```

3. Configuration Guide

Take the RK3506 platform as an example, configure RT-Thread in the `bsp/rockchip/rk3506-32` directory using `scons --menuconfig`. The following configurations are required to enable PWM functionality:

RT-Thread PWM device framework configuration RT_USING_PWM, with the corresponding Help information:

```
Symbol: RT_USING_PWM [=n]
Type  : boolean
Prompt: Using PWM device drivers
  Location:
    -> RT-Thread Components
      -> Device Drivers
  Defined at ../../../components/drivers/Kconfig:131
```

PWM device driver configuration for Rockchip platform RT_USING_PWM0/RT_USING_PWM1, with the corresponding Help information:

```
Symbol: RT_USING_PWM0 [=n]
Type  : boolean
Prompt: Enable PWM0
  Location:
    -> RT-Thread Rockchip RK3506 drivers
      -> Enable PWM
  Defined at ./driver/Kconfig:96
  Depends on: RT_USING_PWM [=n]

Symbol: RT_USING_PWM1 [=n]
Type  : boolean
Prompt: Enable PWM1
  Location:
    -> RT-Thread Rockchip RK3506 drivers
      -> Enable PWM
  Defined at ./driver/Kconfig:100
  Depends on: RT_USING_PWM [=n]
```

4. Software Application

For the application method of RT-Thread PWM, the official documentation "PWM Device" has already provided an explanation. This section mainly briefly introduces the PWM framework and further elaborates on the PWM characteristics of the Rockchip platform.

4.1 Framework

- The RT-Thread PWM framework primarily configures PWM devices through `struct rt_pwm_configuration`:

```
enum rt_pwm_aligned_mode
{
    PWM_LEFT_ALIGNED = 1,
    PWM_CENTER_ALIGNED,
    PWM_UNALIGNED,
};

struct rt_pwm_configuration
{
    rt_uint32_t channel; /* 1-n or 0-n, which depends on specific MCU requirements */
    rt_uint32_t period;  /* unit:ns 1ns~4.29s:1Ghz~0.23hz */
    rt_uint32_t pulse;   /* unit:ns (pulse<=period) */
    rt_uint32_t count;   /* (count+1) repeated effective periods */
    rt_uint32_t offset;  /* unit:ns 0ns<=offset<=(period-duty) */
    rt_uint32_t mask;    /* channel mask for lock/unlock, such as 0x5 indicates channel0 and
channel2 */

    rt_bool_t polarity;  /* polarity inverted or not */
    /*
     * RT_TRUE  : The channel of pwm is complementary.
     * RT_FALSE : The channel of pwm is normal.
    */
    rt_bool_t  complementary;

    rt_uint32_t delay;    /* unit:ms, which indicates timer val in frequency meter mode */
    rt_uint32_t frequency; /* unit:hz, which indicates frequency meter result */

    rt_uint64_t counter_res; /* The number of effective waveforms */

    enum rt_pwm_aligned_mode aligned; /* aligned mode */
};
```

  - The configuration items count/offset/mask/delay/frequency/aligned are newly added for the Rockchip platform.
- The RT-Thread PWM framework provides application interfaces:

```
rt_err_t rt_device_pwm_register(struct rt_device_pwm *device, const char *name, const struct
rt_pwm_ops *ops, const void *user_data);
rt_err_t rt_pwm_enable(struct rt_device_pwm *device, int channel);
rt_err_t rt_pwm_disable(struct rt_device_pwm *device, int channel);
rt_err_t rt_pwm_set(struct rt_device_pwm *device, int channel, rt_uint32_t period, rt_uint32_t
pulse);
rt_err_t rt_pwm_set_period(struct rt_device_pwm *device, int channel, rt_uint32_t period);
rt_err_t rt_pwm_set_pulse(struct rt_device_pwm *device, int channel, rt_uint32_t pulse);
// Rockchip platform newly added
rt_err_t rt_pwm_set_internal(struct rt_device_pwm *device, int channel, rt_uint32_t period,
rt_uint32_t pulse, rt_uint32_t polarity, enum rt_pwm_aligned_mode aligned);
rt_err_t rt_pwm_set_offset(struct rt_device_pwm *device, int channel, rt_uint32_t offset);
rt_err_t rt_pwm_set_oneshot(struct rt_device_pwm *device, int channel, rt_uint32_t count);
rt_err_t rt_pwm_set_capture(struct rt_device_pwm *device, int channel);
rt_err_t rt_pwm_lock(struct rt_device_pwm *device, rt_uint32_t channel_mask);
rt_err_t rt_pwm_unlock(struct rt_device_pwm *device, rt_uint8_t channel_mask);
rt_err_t rt_pwm_int_enable(struct rt_device_pwm *device, int channel);
rt_err_t rt_pwm_int_disable(struct rt_device_pwm *device, int channel);
rt_err_t rt_pwm_counter_enable(struct rt_device_pwm *device, int channel);
rt_err_t rt_pwm_counter_disable(struct rt_device_pwm *device, int channel);
rt_err_t rt_pwm_counter_get_result(struct rt_device_pwm *device, int channel, rt_uint64_t *cnt_res);
rt_err_t rt_pwm_set_frequency_meter(struct rt_device_pwm *device, int channel, rt_uint32_t delay_ms,
rt_uint32_t *freq);
```

  - The above application interfaces on the Rockchip platform are implemented through the rockchip_pwm_control() function in bsp/rockchip/common/drivers/drv_pwm.c combined with each PWM CMD:

```
struct rt_pwm_ops
{
    rt_err_t (*control)(struct rt_device_pwm *device, int cmd, void *arg);
};

#define PWM_CMD_ENABLE      (RT_DEVICE_CTRL_BASE(PWM) + 0)
#define PWM_CMD_DISABLE     (RT_DEVICE_CTRL_BASE(PWM) + 1)
#define PWM_CMD_SET         (RT_DEVICE_CTRL_BASE(PWM) + 2)
#define PWM_CMD_GET         (RT_DEVICE_CTRL_BASE(PWM) + 3)
#define PWMN_CMD_ENABLE     (RT_DEVICE_CTRL_BASE(PWM) + 4)
#define PWMN_CMD_DISABLE    (RT_DEVICE_CTRL_BASE(PWM) + 5)
#define PWM_CMD_SET_PERIOD  (RT_DEVICE_CTRL_BASE(PWM) + 6)
#define PWM_CMD_SET_PULSE   (RT_DEVICE_CTRL_BASE(PWM) + 7)
// Rockchip platform newly added
#define PWM_CMD_SET_ONESHOT (RT_DEVICE_CTRL_BASE(PWM) + 8)
#define PWM_CMD_SET_OFFSET  (RT_DEVICE_CTRL_BASE(PWM) + 9)
#define PWM_CMD_SET_CAPTURE (RT_DEVICE_CTRL_BASE(PWM) + 10)
#define PWM_CMD_LOCK        (RT_DEVICE_CTRL_BASE(PWM) + 11)
#define PWM_CMD_UNLOCK      (RT_DEVICE_CTRL_BASE(PWM) + 12)
#define PWM_CMD_INT_ENABLE  (RT_DEVICE_CTRL_BASE(PWM) + 13)
#define PWM_CMD_INT_DISABLE (RT_DEVICE_CTRL_BASE(PWM) + 14)
#define PWM_CMD_CNT_ENABLE  (RT_DEVICE_CTRL_BASE(PWM) + 15)
#define PWM_CMD_CNT_DISABLE (RT_DEVICE_CTRL_BASE(PWM) + 16)
#define PWM_CMD_GET_CNT_RES (RT_DEVICE_CTRL_BASE(PWM) + 17)
#define PWM_CMD_SET_FREQ    (RT_DEVICE_CTRL_BASE(PWM) + 18)
```

- After enabling the RT_USING_FINSH configuration, you can obtain the Usage information for all test commands through the `pwm` command in the command line:

```
msh />pwm
Usage:
pwm probe   <device name>                                    - probe pwm by name
pwm enable  <channel>                                        - enable pwm channel
pwm disable <channel>                                        - disable pwm channel
pwm get     <channel>                                        - get pwm channel info
pwm set     <channel> <period> <pulse> <polarity> <aligned> - set pwm channel info
pwm offset  <channel> <offset>                               - set pwm channel offset
pwm oneshot <channel> <count>                                - set pwm oneshot mode
pwm capture <channel>                                        - set pwm capture mode
pwm lock    <channel_mask>                                   - enable pwm global lock
pwm unlock  <channel_mask>                                   - disable pwm global lock
pwm int_enable  <channel>                                    - enable pwm interrupt
pwm int_disable <channel>                                    - disable pwm interrupt
pwm cnt_enable  <channel>                                    - enable pwm counter
pwm cnt_disable <channel>                                    - disable pwm counter
pwm cnt_get_res <channel>                                    - get pwm counter result
pwm set_freq    <channel> <mdelay>                           - set pwm frequency meter
```

4.2 Continuous Output Mode

The continuous output mode supports the sustained generation of PWM waveforms with a specified duty cycle.

```
pwm_device = (struct rt_device_pwm *)rt_device_find(name);
result = rt_pwm_set(pwm_device, channel, period, pulse);
/* to config polarity and aligned */
// result = rt_pwm_set_internal(pwm_device, channel, period, pulse, polarity, aligned);
result = rt_pwm_enable(pwm_device, channel);
```

- name: Device name, such as "pwm0", "pwm1", etc.
- channel: Channel ID.
- period: Period, in units of ns.
- pulse: Duty cycle, in units of ns.
- polarity: Waveform polarity, 0 indicates normal, 1 indicates inverted.
- aligned: Alignment mode.

Command line test instructions:

```
pwm probe pwm0
pwm set 0 10000 2000 0 0
pwm enable 0
```

4.3 Oneshot

Single-shot output mode, supports outputting a specified number of PWM waveforms.

```
pwm_device = (struct rt_device_pwm *)rt_device_find(name);
result = rt_pwm_set(pwm_device, channel, period, pulse);
/* to config polarity and aligned */
// result = rt_pwm_set_internal(pwm_device, channel, period, pulse, polarity, aligned);
result = rt_pwm_set_oneshot(pwm_device, channel, count);
result = rt_pwm_enable(pwm_device, channel);
```

- name: Device name, such as "pwm0", "pwm1", etc.
- channel: Channel ID.
- period: Period, unit ns.
- pulse: Duty cycle, unit ns.
- polarity: Waveform polarity, 0 indicates normal, 1 indicates inverted.
- aligned: Alignment mode.
- count: Number of waveforms.

Command line test instructions:

```
pwm probe pwm0
pwm set 0 10000 2000 0 0
pwm oneshot 0 10
pwm enable 0
```

4.4 Capture

Input capture mode, supports calculating the duration of high and low levels of input waveforms.

```
pwm_device = (struct rt_device_pwm *)rt_device_find(name);
result = rt_pwm_int_enable(pwm_device, channel);
result = rt_pwm_set_capture(pwm_device, channel);
```

- name: Device name, such as "pwm0", "pwm1", etc.
- channel: Channel ID.

Command line test commands:

```
pwm probe pwm0
pwm int_enable 0
pwm capture 0
```

4.5 Global Control

Global control mode, which supports synchronous updates for multi-channel configurations, can achieve output synchronization and complementary output functions in conjunction with continuous/oneshot mode.

```
pwm_device = (struct rt_device_pwm *)rt_device_find(name);
result = rt_pwm_lock(pwm_device, channel_mask);
......
result = rt_pwm_unlock(pwm_device, channel_mask);
```

- name: Device name, such as "pwm0", "pwm1", etc.
- channel_mask: Channel mask, such as 0x3 indicating channels channel0 and channel1.

Command line test instructions:

```
pwm probe pwm0
pwm lock 3
pwm set 0 10000 2000 0 0
pwm set 1 10000 2000 0 0
pwm oneshot 0 10
pwm oneshot 1 10
pwm unlock 3
```

4.6 Output Offset

Output offset mode supports the shifting of PWM output waveforms by a specified time, commonly used in conjunction with global control in oneshot mode to achieve more complex waveform output capabilities.

```
pwm_device = (struct rt_device_pwm *)rt_device_find(name);
...
result = rt_pwm_set_offset(pwm_device, channel, offset);
...
```

- name: Device name, such as "pwm0", "pwm1", etc.
- channel: Channel ID.
- offset: Offset time, in ns.

Command line test instructions:

```
pwm probe pwm0
pwm set 0 10000 2000 0 0
pwm offset 0 1000
pwm oneshot 0 10
pwm enable 0
```

4.7 Counter

Input counting mode, supports calculating the number of input waveforms.

```
pwm_device = (struct rt_device_pwm *)rt_device_find(name);
result = rt_pwm_counter_enable(pwm_device, channel);
result = rt_pwm_counter_get_result(pwm_device, channel, &cnt_res);
result = rt_pwm_counter_disable(pwm_device, channel);
```

- name: Device name, such as "pwm0", "pwm1", etc.
- channel: Channel ID.
- cnt_res: Waveform count result.

Command line test commands:

```
pwm probe pwm0
pwm cnt_enable 0
pwm cnt_get_res 0
pwm cnt_disable 0
```

4.8 Frequency Meter

Frequency meter mode, supports calculating the frequency of the input waveform.

```
pwm_device = (struct rt_device_pwm *)rt_device_find(name);
result = rt_pwm_set_frequency_meter(pwm_device, channel, delay_ms, &freq);
```

- name: Device name, such as "pwm0", "pwm1", etc.
- channel: Channel ID.
- delay_ms: Frequency measurement delay time, unit ms.
- freq: Frequency calculation result, unit Hz.

Command line test commands:

```
pwm probe pwm0
pwm set_freq 0 1000
```

4.9 IR Input

For details, refer to the driver `bsp/rockchip/common/drivers/drv_pwm_remotectl.c` and related documentation.